

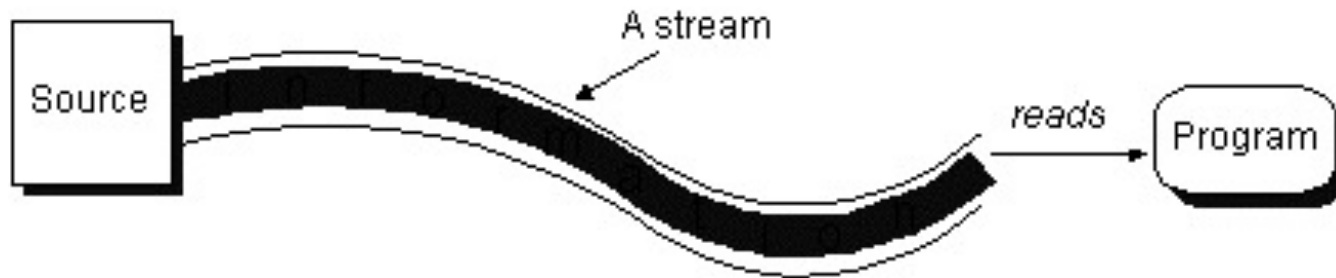
Java

Il package java.io

Lettura e scrittura di dati

- I dati possono provenire/essere inviati su molte sorgenti/destinazioni
 - Memoria
 - Dischi
 - Rete
- In ogni caso uno stream sorgente/stream destinazione deve essere aperto per leggere/scrivere dati.

Lettura & Scrittura dati



Lettura

Apertura di uno Stream

Lettura di più
informazioni

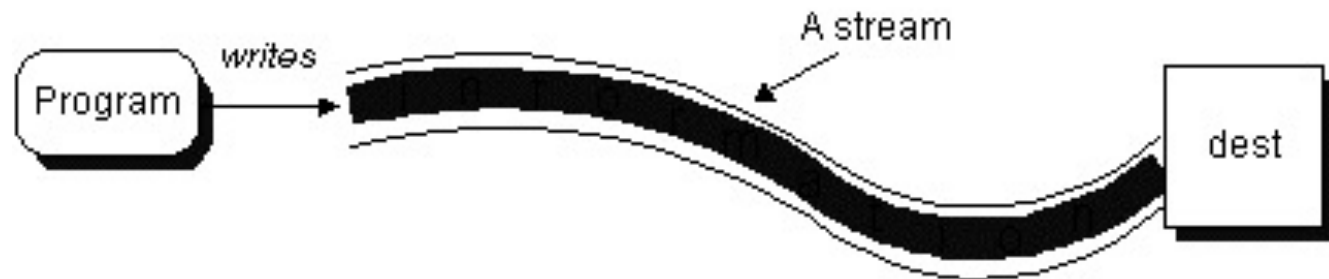
Chiusura dello Stream

Scrittura

Apertura di uno Stream

Scrittura più
informazioni

Chiusura dello Stream



Flussi e canali

- I flussi permettono operazioni di I/O bloccanti (`java.io`)
- I canali permettono operazioni di I/O oltre che bloccanti anche operazioni non bloccanti (`java.nio`)
- Esiste anche `java.net`, che consente l'utilizzo dei socket ed è basato su flussi o canali

IL PACKAGE `java.io`

- Il package `java.io` definisce i concetti base per gestire l'I/O da qualsiasi sorgente e verso qualsiasi destinazione.
- L'obiettivo è fornire
 - un'astrazione che incapsuli tutti i dettagli di una sorgente dati o di un dispositivo di output
 - un modo semplice e flessibile per aggiungere ulteriori funzionalità a quelle fornite dal canale "base"

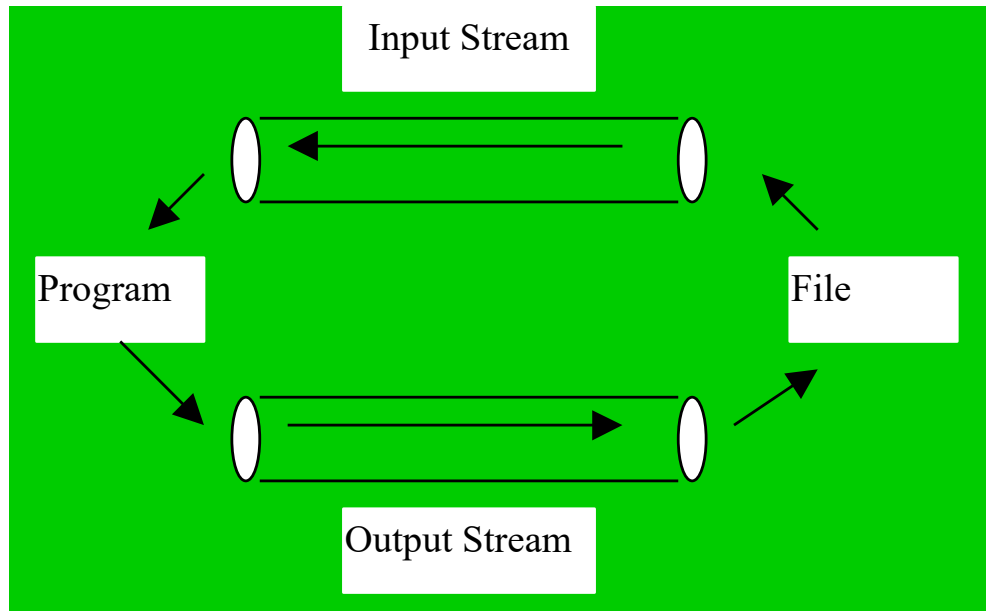
Alcune I/O Exception

Exception class	Categoria di eccezioni
IOException	Qualsiasi eccezione di input/output
EOFException	Eccezione di end-of-file causata da tentative di lettura oltre l'ultimo record del file
FileNotFoundException	Errore causato dal non ritrovamento del file

Streams o flussi

- **Uno stream è un flusso di comunicazione**

- monodirezionale (o di input, o di output)
- di uso generale
- adatto a trasferire byte (o anche caratteri)



Buffer: depositi di dati da cui si può leggere o su cui si può scrivere

Classi stream

- Il package `java.io` distingue fra:
 - stream di byte (analoghi ai file binari del C)
 - stream di caratteri (analoghi ai file di testo del C) (solo da Java 1.1 in poi)
- Questi concetti si traducono in altrettante classi base astratte:
 - stream di byte: `InputStream` e `OutputStream`
 - stream di caratteri: `Reader` e `Writer`
- Le classi `InputStream/OutputStream` sono la radice di tutte le classi di byte e le classi `Reader/Writer` sono la radice di character stream

Text vs. Binary files

- ▣ Numero : 127 (decimal)

- ▣ Text file

- ▣ Three bytes: "1", "2", "7"

- ▣ ASCII (decimal): 49, 50, 55

- ▣ ASCII (octal): 61, 62, 67

- ▣ ASCII (binary): 00110001, 00110010, 00110111

- ▣ Binary file:

- ▣ One byte (byte): 01111110

- ▣ Two bytes (short): 00000000 01111110

- ▣ Four bytes (int): 00000000 00000000 00000000 01111110

Classi stream

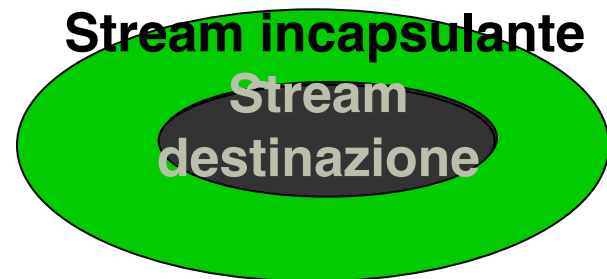
- Le classi e le interfacce di java.io si possono dividere in 5 categorie:
 - Classi generali che permettono di costruire tipi differenti di flussi di byte o di caratteri: flussi di input e flussi di output di byte e caratteri, oltre le classi per convertire gli uni negli altri
 - Un insieme di classi che definiscono varianti di tipi di flussi: filtrati, bufferizzati
 - Un insieme di classi e interfacce riferiti a flussi di dati che permettono di leggere e scrivere valori primitivi
 - Le classi e le interfacce che consentono di interagire con i file
 - Le classi e le interfacce che consentono la serializzazione degli oggetti: cioè che permettono di trasformare gli oggetti in flussi di dati o viceversa

IL PACKAGE `java.io`

- L'approccio "a cipolla"

- alcuni tipi di stream rappresentano sorgenti di dati o dispositivi di uscita

- file, connessioni di rete,
 - array di byte, ...

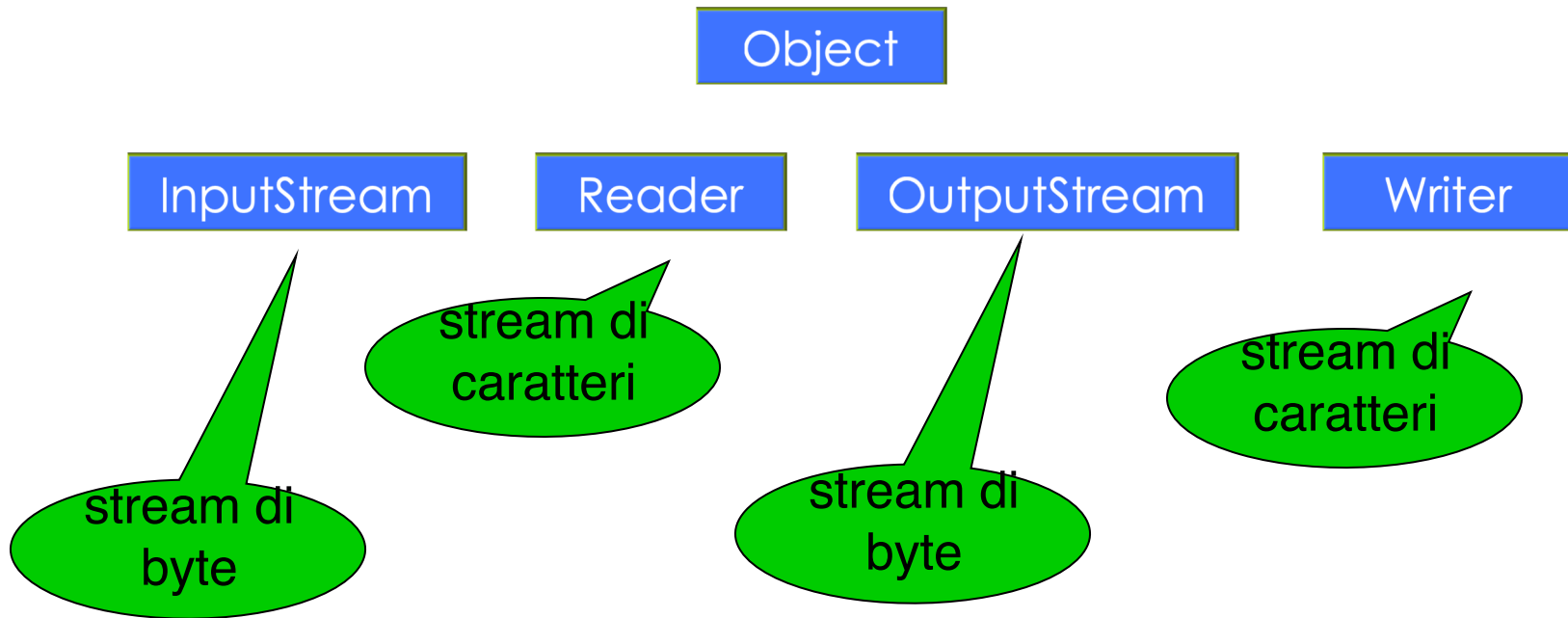


- gli altri tipi di stream sono pensati per "avvolgere" i precedenti per aggiungere ulteriori funzionalità.

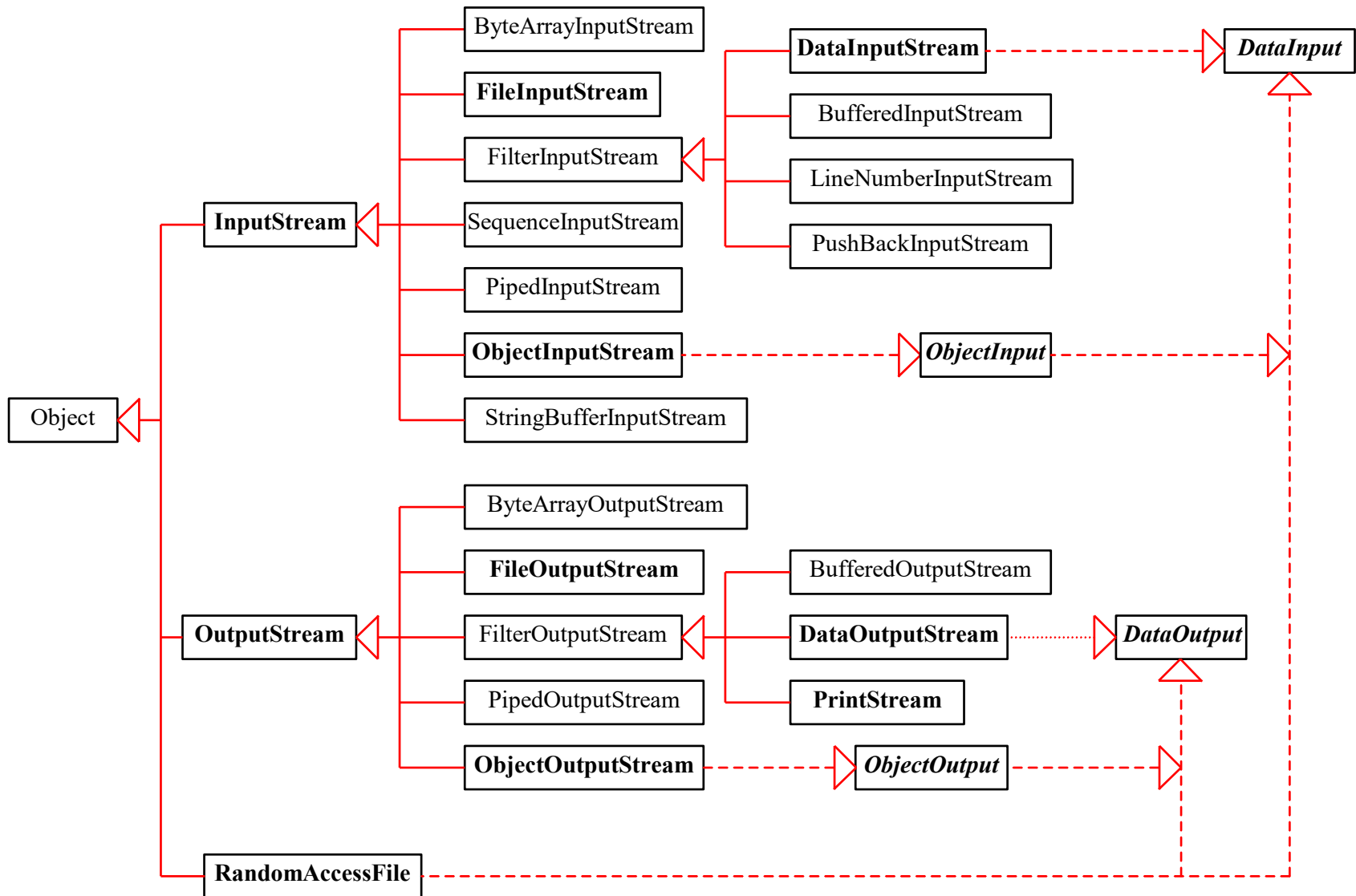
- Così, è possibile configurare il canale di comunicazione con tutte e sole le funzionalità che servono... ..senza peraltro doverle replicare e reimplementare più volte.

IL PACKAGE `java.io`

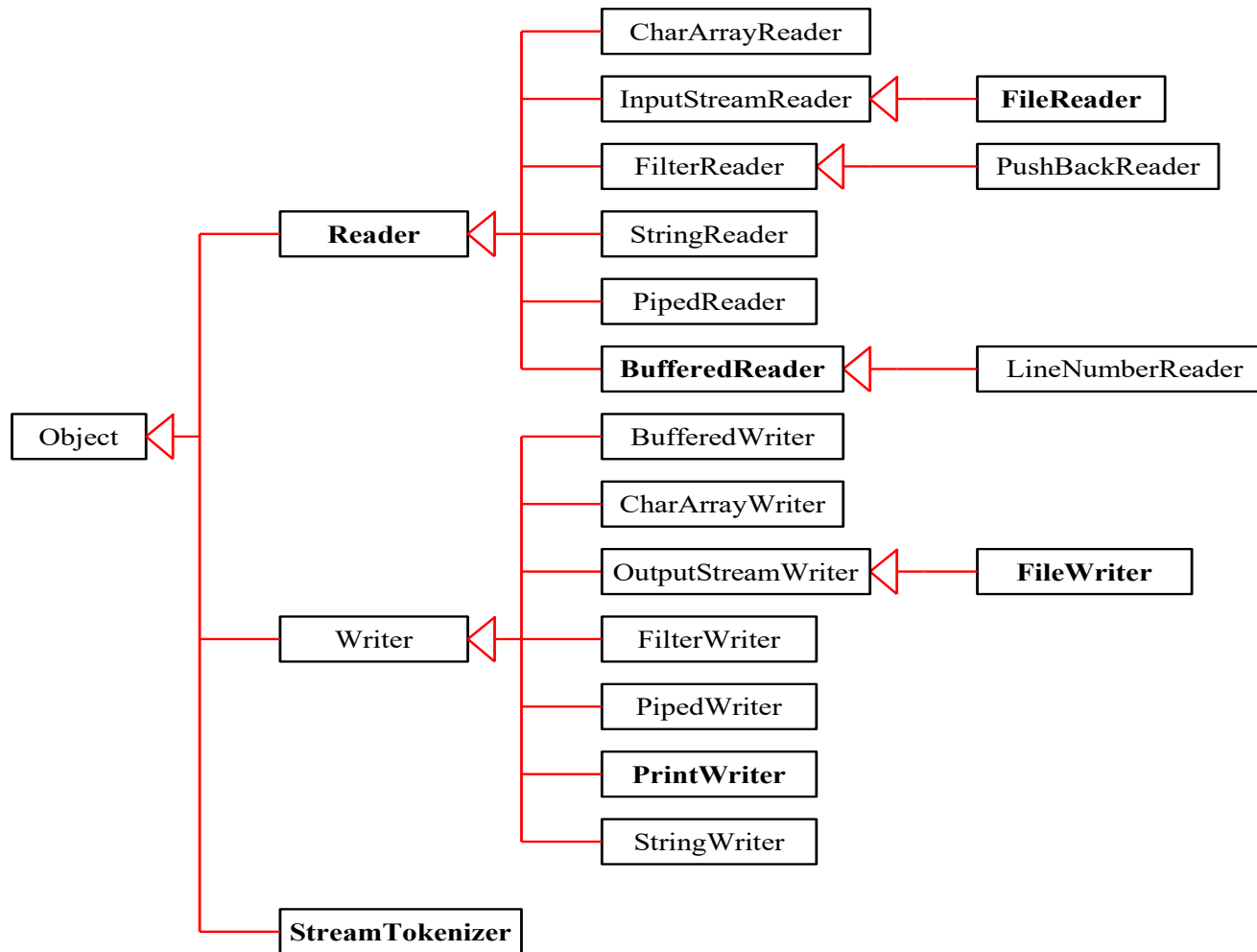
- Le quattro classi base astratte di `java.io`



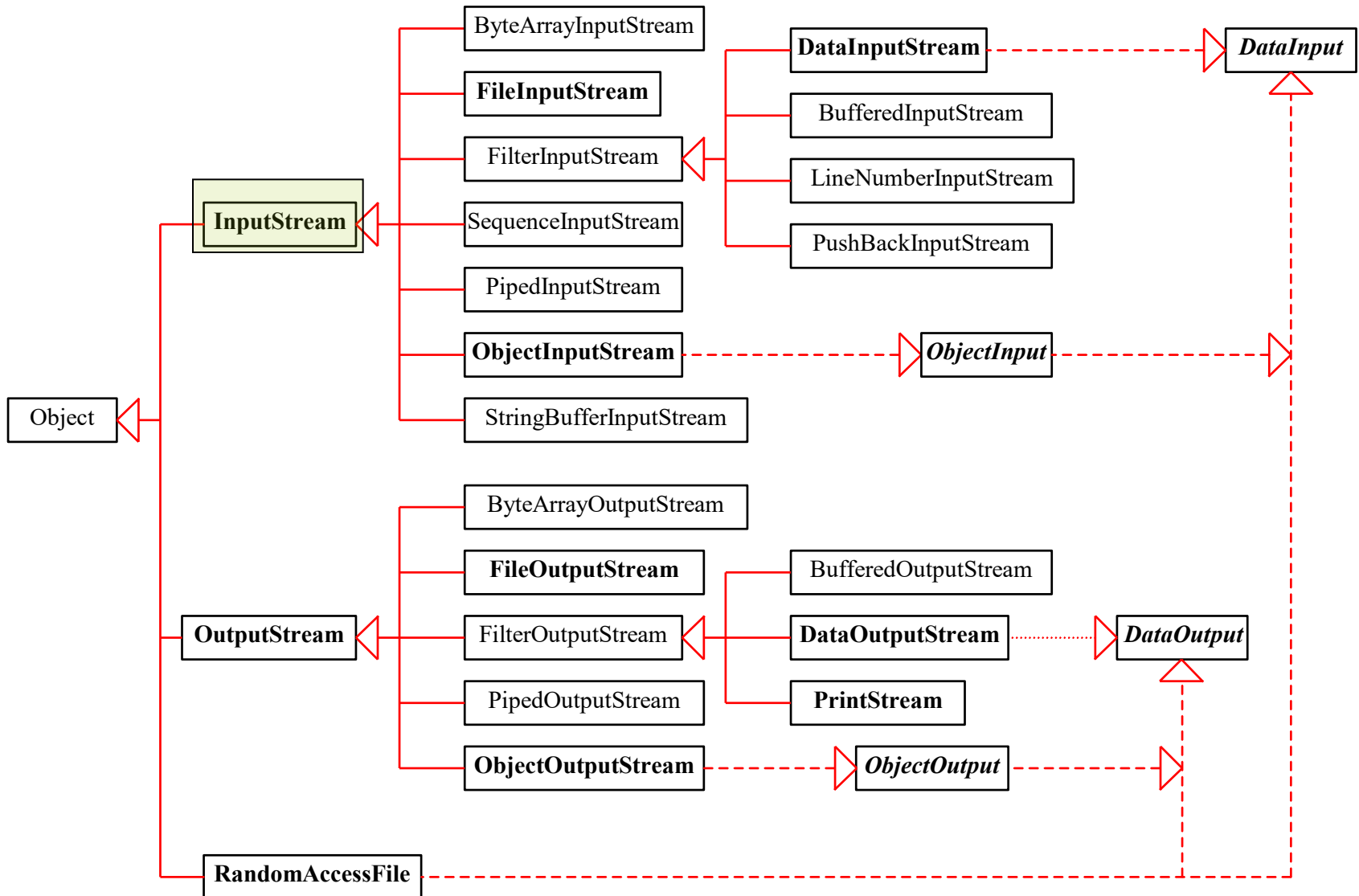
Byte Stream Classes



Character Stream Classes



Byte Stream Classes



InputStream

- E' una classe astratta
- Definisce il concetto generale di “flusso di input” operante sui byte
- Dichiarare i metodi che permettono di leggere i byte a partire da una specifica sorgente
- I metodi dovranno essere realmente definiti dalle classi derivate, in modo specifico alla specifica sorgente dati.
- Il metodo costruttore apre lo stream

Input stream

public abstract int read() throws IOException

- Legge un singolo byte di dati e lo restituisce sotto forma di intero compreso tra 0 e 255. Se non è disponibile alcun byte restituisce il valore -1

**public int read(byte[] b, int offset, int count)
throws IOException**

- Legge una sequenza di byte e la memorizza in una parte di un array di byte a partire dall'elemento di posizione offset e in numero massimo di count

public int read(byte[] b) throws IOException

- Come il metodo precedente con offset=0 e count=buff.length

Input stream

```
public long skip(long count) throws IOException
```

- Avanza il flusso saltando di un numero di byte pari a count. Si arresta se raggiunge la fine del flusso. Restituisce il numero effettivo di byte. Se count è negativo non avanza affatto

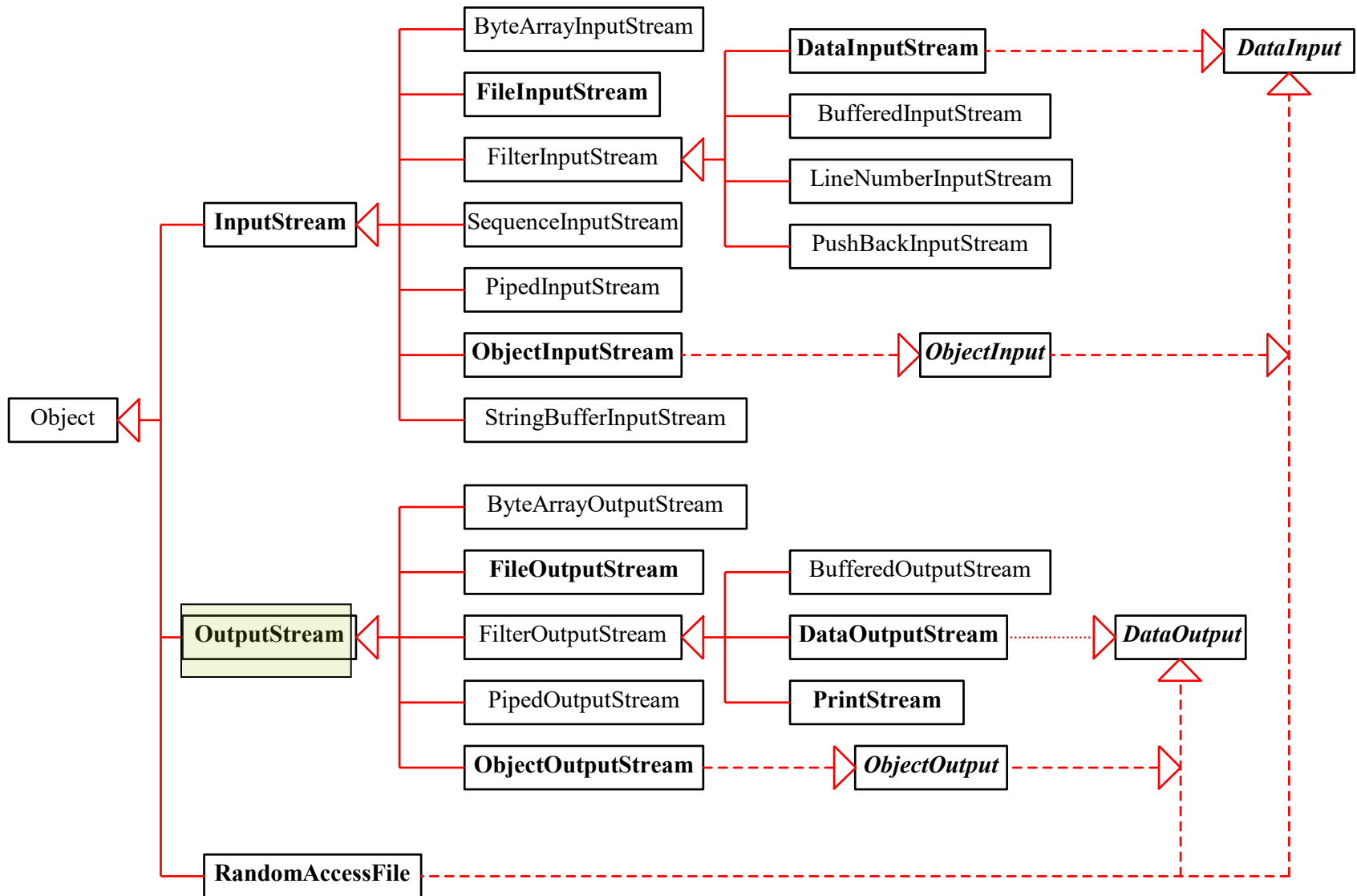
```
public int available() throws IOException
```

- Restituisce il numero di byte che possono essere letti senza arrestarsi.

```
public void close() throws IOException
```

- Chiude il flusso. La chiusura di un flusso già chiuso non ha alcun effetto.

Byte Stream Classes



OutputStream

- E' una classe astratta
- Definisce il concetto generale di “flusso di output” operante sui byte
- Dichiarare i metodi che permettono di scrivere dei byte a partire da una specifica sorgente
- I metodi dovranno essere realmente definiti dalle classi derivate, in modo specifico alla specifica sorgente dati.
- Il metodo costruttore apre lo stream

Output stream

```
public abstract void write(int b) throws IOException
```

- ▣ Scrive b sotto forma di byte. Del valore int b vengono presi in considerazione solo gli otto bit meno significativi

```
public void write(byte[] b, int offset, int count)  
            throws IOException
```

- ▣ Scrive una parte dell'array di byte buf a partire da buf[offset], scrivendo un numero di byte pari a count

```
int write(byte[] b) throws IOException
```

- ▣ Come il metodo precedente con offset=0 e count=buf.length

Output stream

```
public void flush() throws IOException
```

- Effettua lo svuotamento del flusso

```
void close() throws IOException
```

- Chiude il flusso. La chiusura di un flusso già chiuso non ha alcun effetto

Stream di caratteri

- Le classi per l'I/O da stream di caratteri (`Reader` e `Writer`) sono più efficienti di quelle a byte
 - hanno nomi analoghi e struttura analoga
 - Convertono correttamente la codifica UNICODE di Java in quella locale
 - specifica della piattaforma in uso (Windows, DOS, ASCII)...
 - ...e della lingua in uso (essenziale per l'internazionalizzazione).

Reader

La classe `Reader` è simile alla classe `InputStream`. I metodi in `Reader` sono soggetti alla interpretazione dei caratteri.

```
public int read() throws IOException
```

- Legge un carattere e lo restituisce come un intero compreso tra 0 e 65535. Se si legge oltre la fine del flusso restituisce -1

```
public abstract int read(char [] buf, int offset, int count)  
throws IOException
```

- Legge una sequenza di caratteri e lo memorizza in una parte di un array di `char` a partire dall'elemento di posizione `offset` e in numero massimo di `count`

```
public abstract int read(char buf[]) throws IOException
```

- Come il metodo precedente con `offset=0` e `count=buf.length`

Reader

`public void skip(long count) throws IOException`

- Avanza il flusso saltando di un numero di caratteri pari a count. Si arresta se raggiunge la fine del flusso. Restituisce il numero effettivo di byte. Se count è negativo non avanza affatto.

`public boolean ready() throws IOException`

- Restituisce true se il flusso è pronto per essere letto.

`public void close() throws IOException`

- Chiude il flusso. La chiusura di un flusso già chiuso non ha alcun effetto.

Writer

La classe Writer è simile alla classe the OutputStream.

```
public void write (char b) throws IOException
```

- Scrive b sotto forma di carattere.

```
public abstract int write(char [] buf, int offset, int  
count) throws IOException
```

- Scrive una parte di un array di caratteri che comincia con buf[offset] ed è pari a count.

```
public abstract int write(char buf[]) throws IOException
```

- Come il metodo precedente con offset=0 e count=buf.length.

Writer

```
public abstract void flush() throws IOException
```

- Effetua lo svuotamento del flusso.

```
public void close() throws IOException
```

- Chiude il flusso. La chiusura di un flusso già chiuso non ha alcun effetto.

Conversione flussi

- I flussi di conversione `InputStreamReader` e `OutputStreamWriter` permettono di convertire tra loro flussi di caratteri e byte utilizzando una codifica specifica o la codifica di default definita per il sistema locale
- Costruttori
 - `public InputStreamReader(InputStream in)`
 - `public InputStreamReader(InputStream in, Charset c)`
 - `public InputStreamReader(InputStream in, CharserDecoder c)`
 - `public InputStreamReader(InputStream in, String enc) throws UnsupportedOperationException`

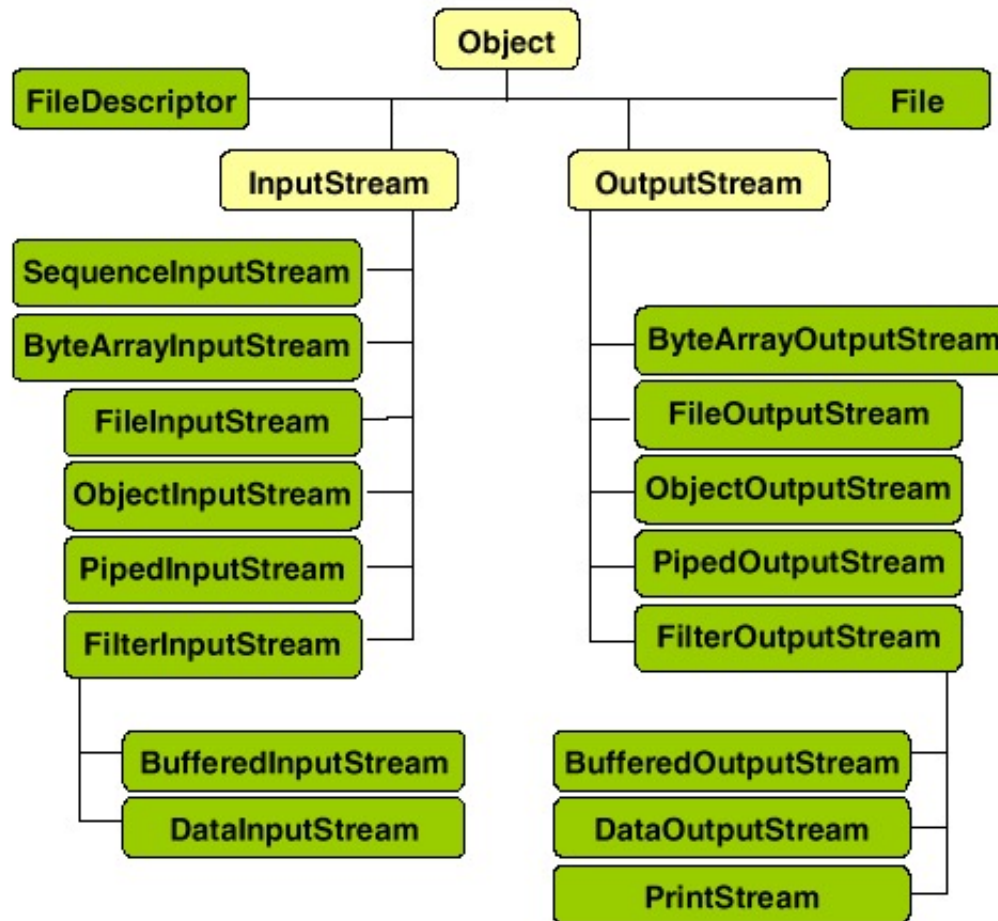
InputStreamReader e OutputStreamWriter

■ Costruttori

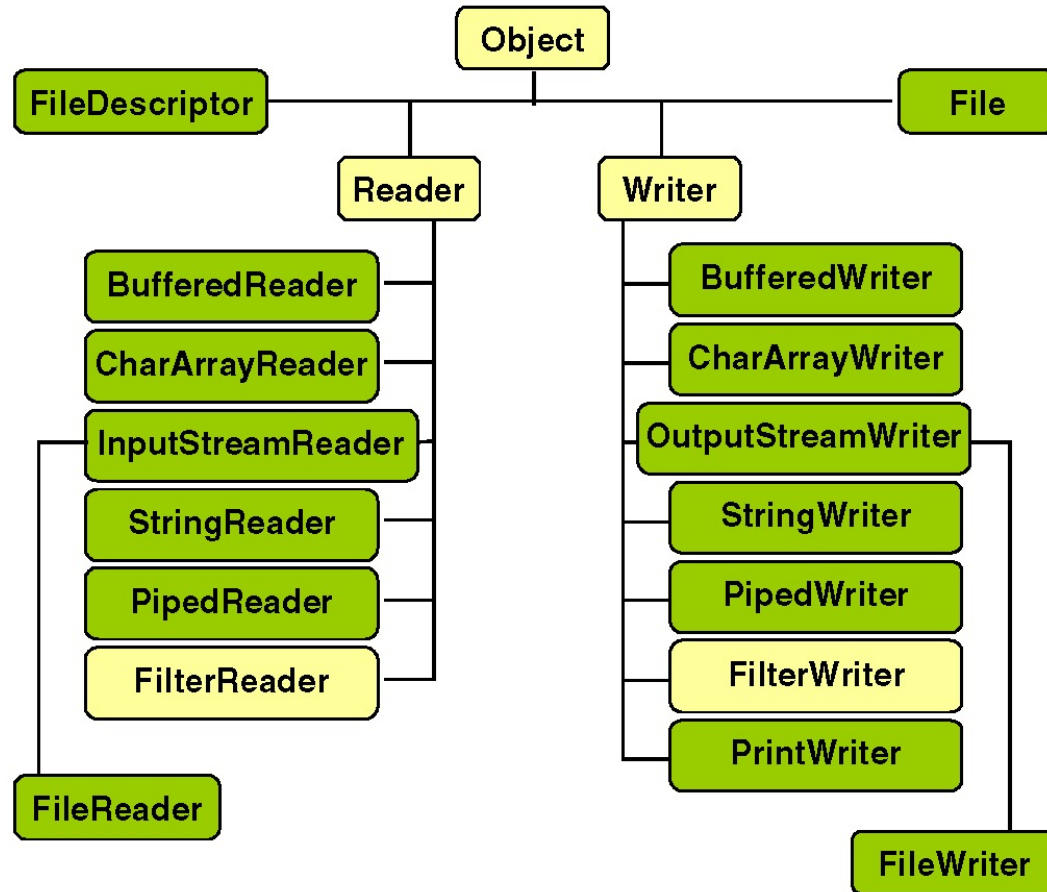
- `public OutputStreamWriter (OutputStream in)`
- `public OutputStreamWriter (OutputStream in, Charset c)`
- `public OutputStreamWriter (OutputStream in, CharserDecoder c)`
- `public OutputStreamWriter (OutputStream in, String enc)`
throws `UnsupportedEncodingException`

- I metodi `read` di `InputStreamReader` leggono i byte dall'`InputStream` assegnato, e li convertono in caratteri con la codifica appropriata; analogamente, i metodi `write` di `OutputStreamReader` converte i caratteri a disposizione in byte e li manda all'`OutputStream` assegnato. In entrambe le classi, la chiusura del flusso di conversione comporta la chiusura del flusso di byte associato.

Stream di byte



Stream di caratteri



Panoramica delle classi dei flussi

- Flussi Filter sono classi astratte che rappresentano tutti quei flussi ai quali è applicata una operazione di flusso
- Flussi Buffered aggiungono ai flussi usuali la presenza di un buffer in modo che read e write non richiedano di accedere al file system ad ogni invocazione
- Flussi piped vengono progettati come coppie

Panoramica delle classi dei flussi

- Flussi di memoria
 - ByteArray
 - CharArray
 - String
- Flussi solo di input o output
 - Print
 - LineNumberReader
 - SequenceInputStream
- Flussi per la costruzione di parser
 - Flussi Pushback
 - StreamTokenizer

Classi dei flussi

Sincronizzazione e concorrenza

- La presenza di diverse classi che operano sui flussi richiede un meccanismo (possibilmente uniforme) di trattamento dei problemi di sincronizzazione e concorrenza
- Le classi che operano sui flussi di byte si sincronizzano su tale flusso, garantendo comunque l'atomicità delle operazioni, ad esempio in caso di scrittura simultanea da parte di due thread, il flusso risultante evita l'inframezzarsi dei byte scritti dai due thread, garantendo che si trovi prima l'intera sequenza di un thread e dopo l'intera sequenza dell'altro. Nell'operazione di lettura di N byte, i due thread accedrebbero a due porzioni di N byte poste consecutivamente nel flusso

Classi dei flussi

Sincronizzazione e concorrenza

- Le classi che operano sui flussi di caratteri si sincronizzano su un campo protetto di nome `lock` che per default è un riferimento al flusso stesso, e che potrebbe anche essere un altro flusso (passabile come parametro dei metodi costruttori di `Reader` e `Writer`), ad esempio la classe `StringWriter`, sottoclasse di `Writer`, scrive i caratteri utilizzando un oggetto `StringBuffer`, a cui `lock` viene riferito; in generale, `lock` deve quindi essere riferito all'oggetto effettivamente usato come flusso

Classi dei flussi

Sincronizzazione e concorrenza

- Nel caso si utilizzino flussi incapsulati, in accordo con l'approccio “a cipolla”, la sincronizzazione dipende dal flusso più interno; questo potrebbe comportare problemi nel caso le classi incapsulanti richiedano operazioni atomiche al loro livello, operazioni che potrebbe non essere garantito che lo siano perché ad esempio potrebbero corrispondere a più operazioni del flusso interno, ciascuna atomica ma non atomiche nel loro insieme

Classi dei flussi

Sincronizzazione e concorrenza

- Le operazioni di input in genere sono bloccanti (si resta in attesa finchè i dati non diventano disponibili), e spesso lo sono anche quelle di output (la scrittura su un socket potrebbe dovere attendere).
- Se queste operazioni, come solitamente si fa, sono implementate da un thread, si potrebbe implementare anche la parte di codice relativa ad una richiesta di interruzione invocata sul thread stesso, ad esempio per liberare la risorsa o per dare informazioni sullo stato di avanzamento prima della richiesta di interruzione. Le situazioni che si dovrebbero fronteggiare sono tuttavia spesso complicate, pertanto si preferisce in genere mantenere completamente bloccanti (non interrompibili) le operazioni di I/O; java.nio offre strumenti aggiuntivi che permettono di affrontare il problema.

Flussi buffered

- Java introduce stream bufferizzati che velocizzano le operazioni di input e output riducendo il numero di letture e scritture.
- Input: un gruppo di dati viene letto tutto in una volta. `BufferedInputStream` e `BufferedReader`
- Output: i dati sono prima chached in un buffer quindi scritti tutti insieme. `BufferedOutputStream` e `BufferedWriter`
- Ogni flusso buffered supporta due costruttori:
 - Uno che richiede un riferimento al flusso incapsulato e le dimensioni del buffer da utilizzare
 - Un secondo richiede solo il riferimento al flusso incapsulato e utilizza una dimensione del buffer di default

Buffered Stream Constructors

- `BufferedInputStream (InputStream in)`
- `BufferedInputStream (InputStream in, int bufferSize)`
- `BufferedOutputStream (OutputStream in)`
- `BufferedOutputStream (OutputStream in, int bufferSize)`
- `BufferedReader (Reader in)`
- `BufferedReader (Reader in, int bufferSize)`
- `BufferedWriter (Writer out)`
- `BufferedWriter (Writer out, int bufferSize)`

StringReader e StringWriter

■ StringReader

- legge i caratteri a partire da una String e non si blocca mai.
- fornisce un unico costruttore che richiede la String a partire dalla quale effettuare la lettura

■ StringWriter

- Permette di scrivere i risultati all'interno di un buffer che può essere recuperato come String o StringBuffer

Flussi Print

I flussi Print permettono di semplificare la scrittura/lettura dei valori appartenenti ai tipi e agli oggetti primitivi.

Forniscono i metodi `print` e `println` per i seguenti:

■ `Object`, `char`, `int`, `float`, `boolean`, `char[]`, `long`, `double` e `String`

Le classi `PrintStream` e `PrintWriter` forniscono queste funzionalità.

PrintWriter Constructors

```
public PrintWriter(Writer out, boolean autoFlush)
```

- Crea un nuovo `PrintWriter` che scriverà sul flusso `out`. Se `autoFlush = true` `println` invocherà `flush()`

```
public PrintWriter(Writer out)
```

- Come il precedente per `autoFlush = false`

```
public PrintWriter(OutputStream out, boolean autoFlush)
```

- E' equivalente a `PrintWriter(new OutputStreamWrite(out))`

```
public PrintWriter(File file) throws FileNotFoundException
```

- E' equivalente a `PrintWriter(new OutputStreamWrite(fos))` dove `fos` è un `FileOutputStream` creato mediante il file assegnato

PrintWriter Methods

- ❑ `void print(Object o)`
- ❑ `void print(String s)`
- ❑ `void println(String s)`
- ❑ `void print(char c)`
- ❑ `void print(char[] cArray)`
- ❑ `void print(int i)`
- ❑ `void print(long l)`
- ❑ `void print(float f)`
- ❑ `void print(double d)`
- ❑ `void print(boolean b)`

DataInput e DataOutput

- Gli stream di dati (`DataInputStream` e `DataOutputStream`) leggono e scrivono tipi primitivi in modo “machine-independent”
- Ciò consente di scrivere un file in una computer e leggerlo in un altro anche con caratteristiche diverse (sistema operativo, struttura del file diversa).
- Rappresentano una implementazione dell'interfaccia `DataInput` e `DataOutput` rispettivamente

DataInputStream Methods

- `int readByte() throws IOException`
- `int readShort() throws IOException`
- `int readInt() throws IOException`
- `int readLong() throws IOException`
- `float readFloat() throws IOException`
- `double readDouble() throws IOException`
- `char readChar() throws IOException`
- `boolean readBoolean() throws IOException`
- `String readUTF() throws IOException`

DataOutputStream Methods

- ❑ `void writeByte(byte b) throws IOException`
- ❑ `void writeShort(short s) throws IOException`
- ❑ `void writeInt(int i) throws IOException`
- ❑ `void writeLong(long l) throws IOException`
- ❑ `void writeFloat(float f) throws IOException`
- ❑ `void writeDouble(double d) throws IOException`
- ❑ `void writeChar(char c) throws IOException`
- ❑ `void writeBoolean(boolean b) throws IOException`
- ❑ `void writeBytes(String l) throws IOException`
- ❑ `void writeChars(String l) throws IOException`
- ❑ `void writeUTF(String l) throws IOException`

Data I/O Stream Constructors

- `DataInputStream in = new DataInputStream(inputstream);`
- `DataInputStream infile = new
 DataInputStream(new FileInputStream("in.dat"));`

Crea un “data input stream” per il file “in.dat”.

- `DataOutputStream out = new DataOutputStream(outputstream);`
- `DataOutputStream outfile = new
 DataOutputStream(new FileOutputStream("out.dat"));`

Crea un “data output stream” per il file “out.dat”.

Flussi per i file

- FileInputStream, FileOutputStream, FileReader e FileWriter permettono di trattare un file come un flusso; presentano tre costruttori
 - Uno con argomento una String, il nome di un file
 - Un oggetto File
 - Un FileDescriptor

- Oltre gli oggetti File, che rappresentano sostanzialmente flussi collegati a file (il ruolo del file è secondario), esistono:
 - la classe FileDescriptor, usata per avere un nuovo flusso a partire dallo stesso file; occorre tuttavia prestare attenzione a più flussi che insistono sullo stesso file
 - La classe File, distinta dai flussi, che fornisce operazioni per la manipolazione del nome del file: path, esistenza del file, permessi, creazione, cancellazione (anche di directory) ecc.
 - La classe RandomAccessFile, che rispetto ai semplici flussi offre un “cursore” per il posizionamento all’interno del file, più i relativi metodi per la sua gestione (seek, skip ecc.).

Stream di byte - input da file

- **FileInputStream** è la classe derivata che rappresenta il concetto di sorgente di byte agganciata a un file
 - il nome del file da aprire è passato come parametro al costruttore di **FileInputStream**
 - in alternativa si può passare al costruttore un oggetto **File** (o un **FileDescriptor**) costruito in precedenza

Stream di byte - input da file

- Per aprire un file binario in lettura si crea un oggetto di classe `FileInputStream`, specificando il nome del file all'atto della creazione.
- Per leggere dal file si usa poi il metodo `read()` che permette di leggere uno o più byte
 - restituisce il byte letto come intero fra 0 e 255
 - se lo stream è finito, restituisce -1
 - se non ci sono byte, ma lo stream non è finito, rimane in attesa dell'arrivo di un byte.
- Poiché è possibile che le operazioni su stream falliscano per varie cause, tutte le operazioni possono lanciare eccezioni → necessità di `try / catch`

Stream di byte - input da file

```
import java.io.*;
public class LetturaDaFileBinario {
    public static void main(String args[]){
        FileInputStream is = null;
        try {
            is = new FileInputStream(args[0]);
        }
        catch(FileNotFoundException e){
            System.out.println("File non trovato");
            System.exit(1);
        }
        // ... lettura ...
    }
}
```

Stream di byte - input da file

La fase di lettura:

```
...  
try {  
    int x = is.read();  
    int n = 0;  
    while (x>=0) {  
        System.out.print(" " + x); n++;  
        x = is.read();  
    }  
    System.out.println("\nTotale byte: " + n);  
} catch (IOException ex) {  
    System.out.println("Errore di input");  
    System.exit(2);  
}
```

quando lo stream termina,
read() restituisce -1

Stream di byte - input da file

▣ Esempio d'uso:

▣ `C:\temp>java LetturaDaFileBinario
question.gif`

▣ Il risultato:

```
71 73 70 56 57 97 32 0 32 0 161 0 0 0 0 0 255  
255 255 ..... .
```

Totale byte: 190

Stream di byte - output su file

- ▣ `FileOutputStream` è la classe derivata che rappresenta il concetto di dispositivo di uscita agganciato a un file
 - ▣ il nome del file da aprire è passato come parametro al costruttore di `FileOutputStream`
 - ▣ in alternativa si può passare al costruttore un oggetto `File` (o un `FileDescriptor`) costruito in precedenza.

Stream di byte - output su file

- Per aprire un file binario in scrittura si crea un oggetto di classe `FileOutputStream`, specificando il nome del file all'atto della creazione
 - un secondo parametro opzionale, di tipo boolean, permette di chiedere l'apertura in modo append
 - Per scrivere sul file si usa il metodo `write()` che permette di scrivere uno o più byte
 - scrive l'intero ($0 \div 255$) passatogli come parametro
 - non restituisce nulla
- Poiché è possibile che le operazioni su stream falliscano per varie cause, tutte le operazioni possono lanciare eccezioni → necessità di `try / catch`

Stream di byte - output su file

```
import java.io.*;
public class ScritturaSuFileBinario {
    public static void main(String args[]){
        FileOutputStream os = null;
        try { os = new FileOutputStream(args[0]);
        }
        catch(FileNotFoundException e){
            System.out.println("Imposs. aprire
file");
            System.exit(1);
        }
        // ... scrittura
    }
}
```

Per aprirlo in modalità append:
`FileOutputStream(args[0],true)`

Stream di byte - output su file

Esempio: scrittura di alcuni byte a scelta

```
...  
try {  
    for (int x=0; x<10; x+=3) {  
        System.out.println("Scrittura di " + x);  
        os.write(x);  
    }  
} catch(IOException ex){  
    System.out.println("Errore di output");  
    System.exit(2);  
}  
...
```

Stream di byte - output su file

- Esempio d'uso:

```
C:\temp>java ScritturaSuFileBinario prova.dat
```

- Il risultato:

```
Scrittura di 0
```

```
Scrittura di 3
```

```
Scrittura di 6
```

```
Scrittura di 9
```

- Controllo:

```
C:\temp>dir prova.dat
```

```
16/01/01 prova.dat 4 byte
```

- Esperimenti

- Provare a rileggere il file con il programma precedente

- Aggiungere altri byte riaprendo il file in modo append

Esempio 1

- **Scrittura di dati su file binario**
 - Per scrivere su un file binario occorre un `FileOutputStream`, che però consente solo di scrivere un byte o un array di byte
 - Volendo scrivere dei `float`, `int`, `double`, `boolean`, ... è molto più pratico un `DataOutputStream`, che ha metodi idonei
 - Si incapsula `FileOutputStream` dentro un `DataOutputStream`

Esempio 1

```
import java.io.*;
public class Esempio1 {
    public static void main(String args[]){
        FileOutputStream fs = null;
        try {
            fs = new FileOutputStream("Prova.dat");
        }
        catch(IOException e){
            System.out.println("Apertura fallita");
            System.exit(1);
        }
        // continua...
```

Esempio 1

```
DataOutputStream os = new DataOutputStream(fs);
float f1 = 3.1415F; char c1 = 'X';
boolean b1 = true; double d1 = 1.4142;
try {
    os.writeFloat(f1); os.writeBoolean(b1);
    os.writeDouble(d1); os.writeChar(c1);
    os.writeInt(12); os.close();
}
catch (IOException e){
    System.out.println("Scrittura fallita");
    System.exit(2);
}
}
```

Esempio 2

- **Rilettura di dati da file binario**
 - Per leggere da un file binario occorre un `FileInputStream`, che però consente solo di leggere un byte o un array di byte
 - Volendo leggere dei `float`, `int`, `double`, `boolean`, ... è molto più pratico un `DataInputStream`, che ha metodi idonei
 - Si incapsula `FileInputStream` dentro un `DataInputStream`

Esempio 2

```
import java.io.*;
public class Esempio2 {
    public static void main(String args[]){
        FileInputStream fin = null;
        try {fin = new FileInputStream("Prova.dat");
        }
        catch(FileNotFoundException e){
            System.out.println("File non
trovato");
            System.exit(3);
        }
    }
// continua...
```

Esempio 2

```
DataStream is = new DataInputStream(fin);
float f2; char c2; boolean b2; double d2;
int i2;
try {
    f2 = is.readFloat(); b2 = is.readBoolean();
    d2 = is.readDouble(); c2 = is.readChar();
    i2 = is.readInt(); is.close();
    System.out.println(f2 + ", " + b2 + ", " + d2 + ", "
                       + c2 + ", " + i2);
} catch (IOException e){
    System.out.println("Errore di input");
    System.exit(4);
}
}
```

Random Access Files

- La classe `RandomAccessFiles` rappresenta un file con un cursore utilizzato per l'accesso diretto
- `RandomAccessFiles`, non è sottoclasse di nessuna `InputStream`, `OutputStream` o `Reader` o `Writer`, in quanto implementa funzionalità sia di input che di output; implementa anche le interfacce `DataInput` e `DataOutput` per i tipi primitivi, oltre che i metodi `read` e `write` delle classi dei flussi; un oggetto flusso non è tuttavia sostituibile con uno `RandomAccessFile`

RandomAccessFile Methods, cont.

- `void seek(long pos)` throws `IOException`;
 - offset dall'inizio del `RandomAccessFile`.
- `long getFilePointer()` `IOException`;
 - Ritorna il corrente offset, in bytes, dall'inizio del file.
- `long length()` throws `IOException`
 - Ritorna la lunghezza del file.
- `void setLength(long newLength)` throws `IOException`
 - Consente di variare la dimensione del file, riempiendolo di byte se più corto o troncandolo se più lungo

RandomAccessFile Methods, cont.

- `final void writeChar(int v) throws IOException`
Scrive un carattere nel file nella sua codifica, con "high byte" scritto prima.
- `final void writeChars(String s) throws IOException`
Scrive una stringa in un file come una sequenza di caratteri.

RandomAccessFile Constructor

```
RandomAccessFile raf =  
    new RandomAccessFile("test.dat", "rw");  
    // read e write
```

```
RandomAccessFile raf =  
    new RandomAccessFile("test.dat", "r"); //read
```

Stream di caratteri

- Cosa cambia rispetto agli stream binari ?
 - Il file di testo si apre costruendo un oggetto `FileReader` o `FileWriter`, rispettivamente
 - `read()` e `write()` leggono/scrivono un `int` che rappresenta un carattere UNICODE
 - ricorda: un carattere UNICODE è lungo due byte
 - `read()` restituisce -1 in caso di fine stream
- Occorre dunque un cast esplicito per convertire il carattere UNICODE in `int` e viceversa

Esempio

```
import java.io.*;

public class LetturaDaFileDiTesto {
    public static void main(String args[]){
        FileReader r = null;

        try {
            r = new FileReader(args[0]);
        }
        catch(FileNotFoundException e){
            System.out.println("File non trovato");
            System.exit(1);
        }

        // ... lettura ...
    }
}
```

Esempio

La fase di lettura:

```
...
try {
    int n=0, x = r.read();
    while (x>=0) {
        char ch = (char) x;
        System.out.print(" " + ch); n++;
        x = r.read();
    }
    System.out.println("\nTotale caratteri: " +
n);
} catch(IOException ex){
    System.out.println("Errore di input");
    System.exit(2);
}
...
```

Cast esplicito da **int** a **char** - Ma solo se è stato davvero letto un carattere (cioè se non è stato letto -1)

Esempio

■ Esempio d'uso:

```
C:\temp>java LetturaDaFileDiTesto prova.txt
```

Il risultato:

```
N e l m e z z o d e l c a m m i n d i n o s t r a v  
i t a
```

Totale caratteri: 35

Analogo esercizio può essere svolto per la scrittura su file di testo.

Altri aspetti

- **Gli stream di byte sono più antichi e di livello più basso rispetto agli stream di caratteri**
 - Un carattere UNICODE è una sequenza di due byte
 - Gli stream di byte esistono da Java 1.0, quelli di caratteri da Java 1.1, **le classi esistenti fin da Java 1.0 usano stream di byte anche quando sarebbe meglio usare stream di caratteri → i caratteri potrebbero non essere sempre trattati in modo coerente** (in applicazioni che operano con diverse codifiche, magari su piattaforme diverse, o via rete)
.
- **Alcuni canali di comunicazione sono stream di byte ma a volte si devono usare per inviare / ricevere caratteri** (caso tipico: stream di rete - si inviano GIF ma anche testo)

Conseguenza

- Occorre spesso poter reinterpretare uno stream di byte come reader / writer quando opportuno (cioè quando trasmette caratteri)
- Esistono due classi "incapsulanti" progettate proprio per questo scopo:
 - `InputStreamReader` che reinterpreta un `InputStream` come un `Reader`
 - `OutputStreamWriter` che reinterpreta un `OutputStream` come un `Writer`

Il caso dell' I/O da console

- Video e tastiera sono rappresentati dai due oggetti statici `System.in` e `System.out`
- Poiché esistono fin da Java 1.0 (quando Reader e Writer non esistevano), essi sono formalmente degli stream di byte...
 - `System.in` è formalmente un `InputStream`
 - `System.out` è formalmente un `OutputStream` ma in realtà sono stream di caratteri.
- Per assicurare che i caratteri UNICODE siano ben interpretati occorre quindi incapsularli rispettivamente in un Reader e in un Writer e specificare il character encoding opportuno.

Il caso dell' I/O da console

- `System.in` può essere “interpretato come un Reader” incapsulandolo dentro a un `InputStreamReader`
- `System.out` può essere “interpretato come un Writer” incapsulandolo dentro a un `OutputStreamWriter`
- Tipicamente:

```
InputStreamReader tastiera = new  
    InputStreamReader(System.in);  
  
OutputStreamWriter video = new  
    OutputStreamWriter(System.out);
```

Console input using System.in

```
import java.io.*;

public class ConsoleInput {

    public static void main(String[] args) {

        try {
            // crea un oggetto ins e lo collega alla console.
            InputStreamReader inStream = new
                InputStreamReader(System.in);
            BufferedReader ins = new BufferedReader(inStream);

            // Legge il nome del file di output.
            System.out.print("Enter output file name: ");
            String outFileNames = ins.readLine();

            // crea un oggetto outs -- output file.
            FileWriter outputStream = new FileWriter(outFileNames);
            PrintWriter outs = new PrintWriter(outputStream);

            // legge il numero di linee del file.
            System.out.println("How many data lines?");
            int numLines = Integer.parseInt(ins.readLine());
```

Console input using System.in, cont'd.

```
// legge le righe dalla console e le scrive sul file di output
System.out.println("Type " + numLines + " lines:");
for (int lineCount = 0; lineCount < numLines; lineCount++)
{
    String dataLine = ins.readLine();
    outs.println(dataLine);
}

// scrive un messaggio sulla console e chiude il file
System.out.println(numLines + "righe inserite nel file"
                  + outFileName);
ins.close();
outs.close();
}
catch (IOException ex) {
    System.out.println("i/o error: " + ex.getMessage());
    ex.printStackTrace();
}
catch (NumberFormatException ex) {
    System.out.println(ex.getMessage());
    ex.printStackTrace();
}
}
```

MS-DOS ConsoleInput

AppAccelerator(tm) 1.2.010 for Java (JDK 1.2), x86 version.
Copyright (c) 1997-1999 Inprise Corporation. All Rights Reserved.
Enter output file name: myOutFile.txt
How many data lines?
3
Type 3 lines:
This is a line.
This is one too.
This is line 3.
3 data lines written to file myOutFile.txt

Object Streams

Object streams consentono di effettuare input e output a livello di oggetto.

Per rendere un oggetto leggibile e scrivibile nella definizione della classe si devono implementare le interfacce

java.io.Serializable

o

java.io.Externalizable.

The Serializable Interface

L'interfaccia Serializable è un costruttore di interfacce. Essa non ha metodi così non è necessario aggiungere codice se una classe implements Serializable.

Implementando questa interfaccia viene abilitato il meccanismo automatico di serializzazione per automatizzare il processo memorizzare oggetti e array

The Object Streams

`ObjectOutputStream`

`ObjectInputStream`

The ObjectOutputStream and ObjectInputStream

